

E2E Automation Demo — SauceDemo

Companion document for the demo video — Automated QC workflow (6 parts)

Project	Project NITRO — QC Automation
Demo site	https://www.saucedemo.com
Tech	Playwright + TypeScript (Page Object Model)
Author	Thong Ho (QC)
Date	2026-07-10

The demo video walks through the full automated QC workflow in 6 parts: generating test cases → writing automation code → running tests → capturing the bug → creating the Jira ticket → re-running to check the bug lifecycle.

Part 1 — Generate test cases

- I use Claude to read the Functional Specs and generate test cases. Since this is a sample project, I use Claude to generate test cases from the SauceDemo.com website.
- Test case link: [Saucedemo_TestCase \(Confluence\)](#)

Part 2 — Write E2E automation code

- I use Claude to write the E2E automation code based on the generated test cases.
- The code follows the Page Object Model: page objects in `pages/saucedemo/`, test files in `tests/e2e/`.

Part 3 — Run E2E tests

- Run with the command:

```
npm run test:saucedemo -- --headed
```

Result:

Total	Passed	Failed
39	38	1

Part 4 — Capture Bug (bug + video + image)

- One case fails on purpose to check whether the bug is captured. Each bug is saved to 3 locations: `bug-report`, `bug-image`, `bug-video`.
- **bug-report** — content to paste into the Description of the Jira ticket: Title, Issue, Steps, Actual result, Expected result, Screenshot, ...
- **bug-image** — image of the issue to attach to the ticket.
- **bug-video** — video of the issue to attach to the ticket.

Part 5 — Create the Bug ticket on Jira

- After running the tests and capturing the bug, bugs are created automatically and pushed to Jira under the Epic set in the .env file (e.g. NIT-5006).
- The ticket type depends on the parent link:
 - If the link is an **Epic** → the ticket is of type **Bug**.
 - If the link is a **Story** → the ticket is a **Bug Fixing (in-sprint) Sub-task**.

Part 6 — Re-run tests (verify the bug lifecycle)

- **Run 1 — no duplicates:** Re-run while the bug still exists, to check the system recognizes the existing bug and does **NOT create a duplicate ticket** — if the ticket already exists it only updates the status.
- **Run 2 — auto-close on pass:** Use Claude to simulate the bug being fixed (test passes), to check the system detects it and transitions the ticket to **Done/Pass**.

Appendix — Test case list (SauceDemo)

The test-case suite generated in Part 1 (referenced on Confluence). 36 core test cases across 5 modules; the demo run adds one intentional fail case to illustrate the bug flow (Total 39).

Module 1 — Login (8)

Scenario ID	Tags	Scenario
TC_LOGIN_001	@ui @smoke @positive	Login successfully with standard_user → inventory page, product list visible
TC_LOGIN_002	@ui @negative	locked_out_user → error "Sorry, this user has been locked out."
TC_LOGIN_003	@ui @negative	Empty username → error "Username is required"
TC_LOGIN_004	@ui @negative	Empty password → error "Password is required"
TC_LOGIN_005	@ui @negative	Invalid credentials → error "Username and password do not match"
TC_LOGIN_006	@ui @positive	Click the X button → error message clears
TC_LOGIN_007	@ui @performance	performance_glitch_user logs in slowly but still reaches inventory (up to ~10s)
TC_LOGIN_008	@ui @smoke @positive	Logout from menu → back to login page, Login button visible

Module 2 — Inventory (10)

Scenario ID	Tags	Scenario
TC_INV_001	@ui @smoke @positive	Inventory page displays exactly 6 products
TC_INV_002	@ui @positive	Sort name A → Z correctly
TC_INV_003	@ui @positive	Sort name Z → A correctly
TC_INV_004	@ui @positive	Sort price low → high correctly
TC_INV_005	@ui @positive	Sort price high → low correctly
TC_INV_006	@ui @smoke @positive	Add 1 product → badge shows "1"
TC_INV_007	@ui @positive	Add multiple products → badge shows "2"
TC_INV_008	@ui @positive	Remove product from inventory → badge disappears
TC_INV_009	@ui @positive	Click product name → detail page
TC_INV_010	@ui @positive	Click cart icon → cart page

Module 3 — Cart (6)

Scenario ID	Tags	Scenario
TC_CART_001	@ui @smoke @positive	Cart page shows the added product correctly (name, description, price)

Scenario ID	Tags	Scenario
TC_CART_002	@ui @positive	Remove product on cart → cart empty, badge disappears
TC_CART_003	@ui @positive	Continue Shopping → back to inventory
TC_CART_004	@ui @smoke @positive	Checkout → go to info step
TC_CART_005	@ui @positive	Empty cart when nothing added
TC_CART_006	@ui @positive	Reload → cart keeps product, badge still "1"

Module 4 — Checkout (11)

Scenario ID	Tags	Scenario
TC_CHK_001	@ui @smoke @positive	Enter valid info → go to step 2
TC_CHK_002	@ui @negative	Empty First Name → error "First Name is required"
TC_CHK_003	@ui @negative	Empty Last Name → error "Last Name is required"
TC_CHK_004	@ui @negative	Empty Zip → error "Postal Code is required"
TC_CHK_005	@ui @positive	Cancel on step 1 → back to cart
TC_CHK_006	@ui @positive	Cancel on step 2 → back to inventory
TC_CHK_007	@ui @positive	Order total is correct: total = subtotal + tax
TC_CHK_008	@ui @positive	Back Home on confirmation → inventory, badge disappears
TC_CHK_009	@ui @positive	Step 2 shows all: items, subtotal, tax, total
TC_CHK_010	@ui @smoke @positive @e2e	Complete checkout — Finish → confirmation screen
TC_CHK_011	@ui @smoke @positive @e2e	Confirmation shows "Thank you for your order!"

Module 5 — Navigation (3)

Scenario ID	Tags	Scenario
TC_NAV_001	@ui @positive	Open/close burger menu
TC_NAV_002	@ui @positive	Reset App State clears cart
TC_NAV_003	@ui @positive	All Items → back to inventory